

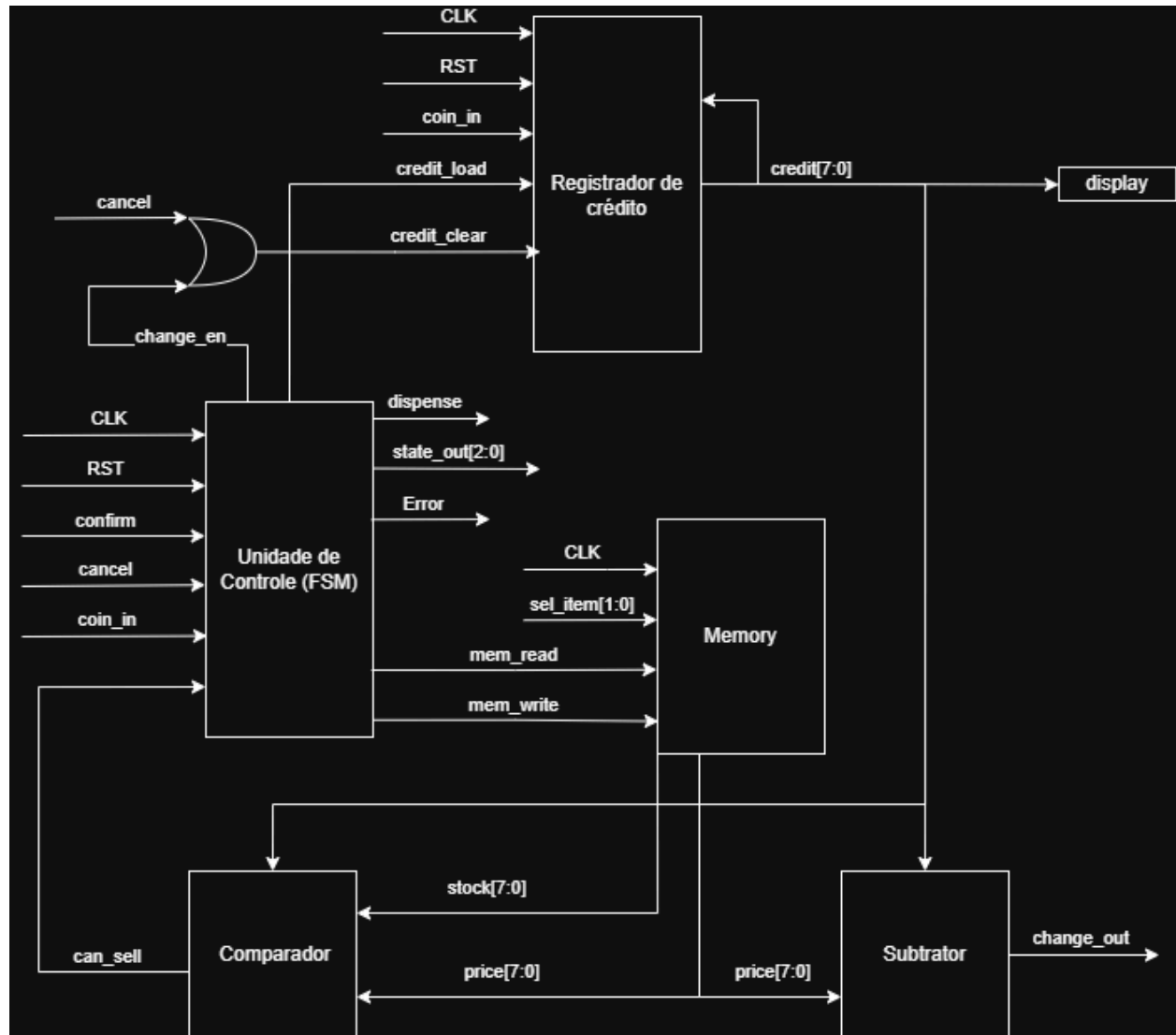
Relatório Técnico: Vending Machine

Aluno: Daniel Lima Neto

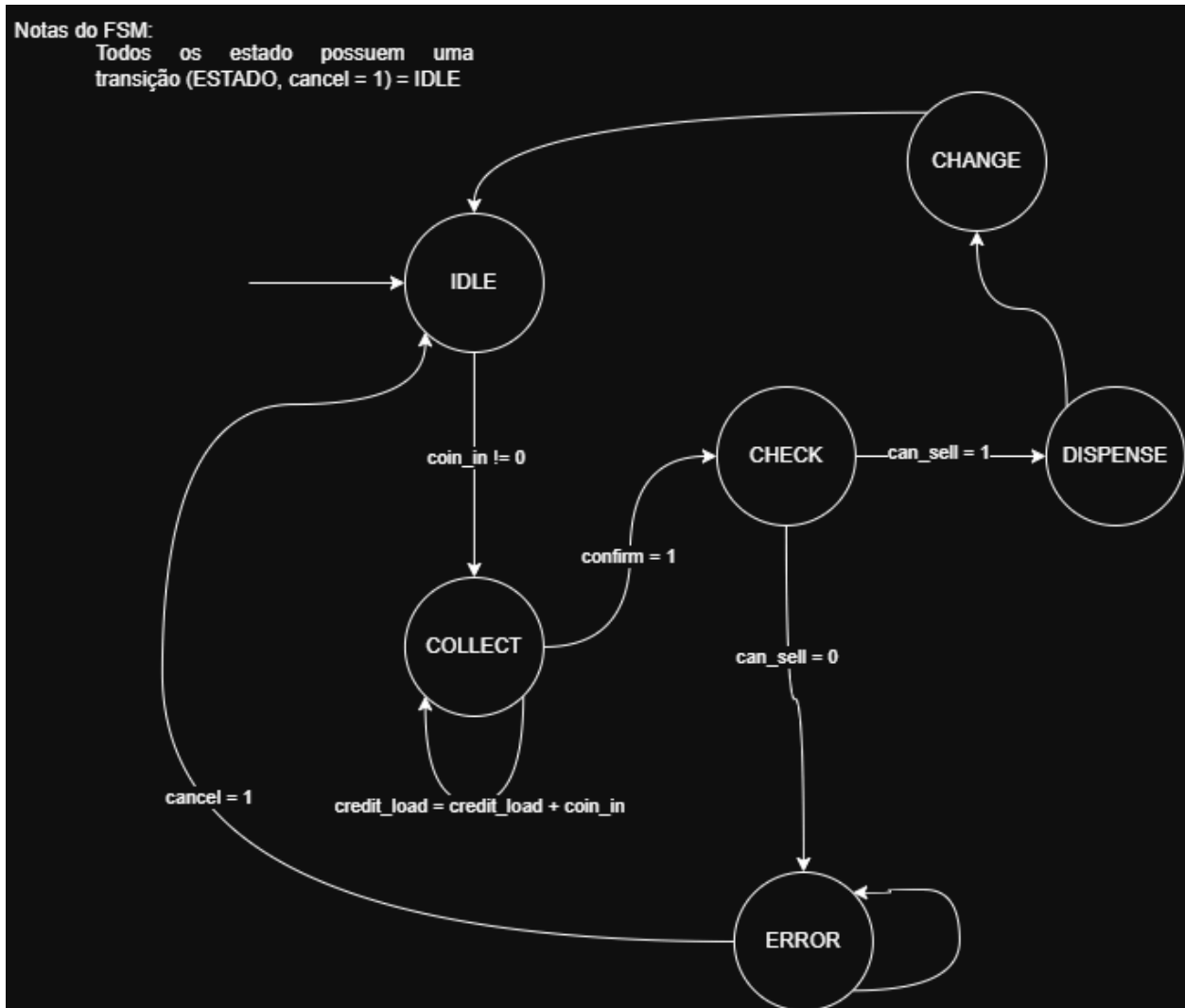
1. Link para o repositório do projeto

https://github.com/Daniellineto/vending_machine.git

2. Diagrama de Blocos



3. Diagrama de Estados



4. Descrição dos Módulos: Decisões de Projeto e Justificativas

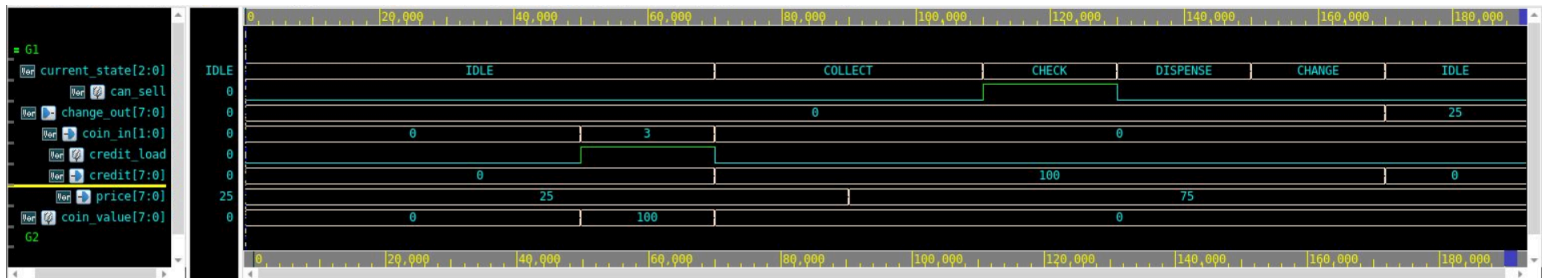
A arquitetura foi dividida rigorosamente entre **Caminho de Dados (Datapath)** e **Unidade de Controlo (FSM)** para garantir modularidade e aderência às boas práticas de RTL Design.

- **vending_pkg**: Criado para centralizar as codificações de estados, valores das moedas e preços, evitando "números mágicos" no código.
- **credit_reg**: Registo com somador combinacional acoplado. Zera quando recebe a instrução ou acumula o valor traduzido da moeda inserida.
- **memory**: Memória síncrona inicializada com stocks específicos por produto (Café e Água com 5 unidades, Suco com 3 e Snack com 2). Gere a leitura de preços e o decremento do stock de forma independente, controlada pelos sinais `mem_read` e `mem_write`.

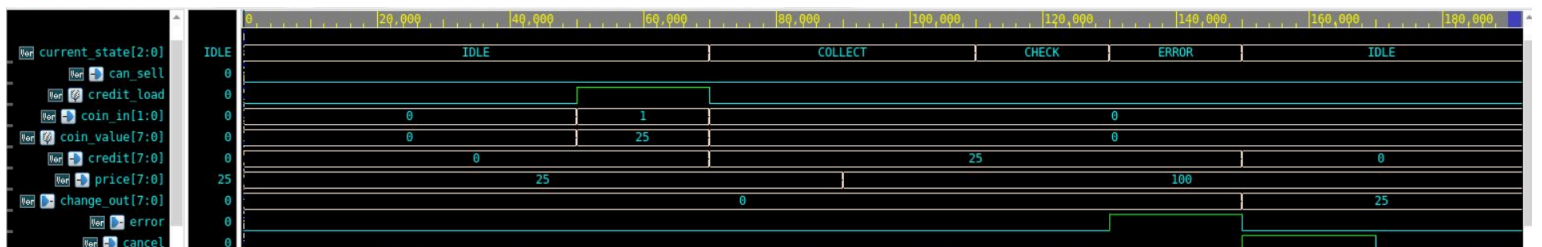
- **comparator & subtractor:** Módulos puramente combinacionais. O comparador avalia se `credit >= price` e `stock > 0`, gerando a flag `can_sell` para a FSM. O subtrator calcula `credit - price` continuamente.
- **unit_control (FSM):** Máquina de Moore responsável exclusivamente pelo controle sequencial. Não manipula dados de 8 bits, apenas emite sinais de habilitação (`credit_load`, `change_en`, etc.) baseados no estado atual.
- **vending_top:** Realiza a interligação estrutural (Top-Level) de todos os submódulos, além de registrar as saídas finais (como `change_out` e `display`) para evitar glitches lógicos.

5. Waveforms Anotadas

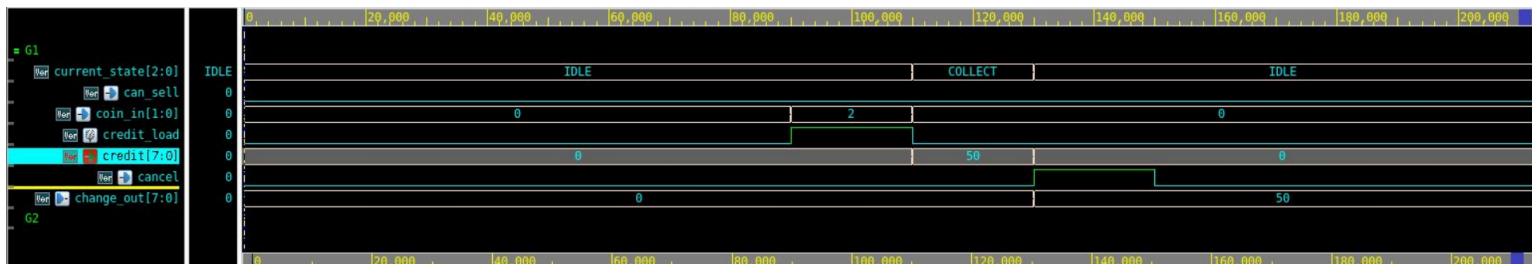
- **Cenário 1: Compra bem-sucedida com troco**



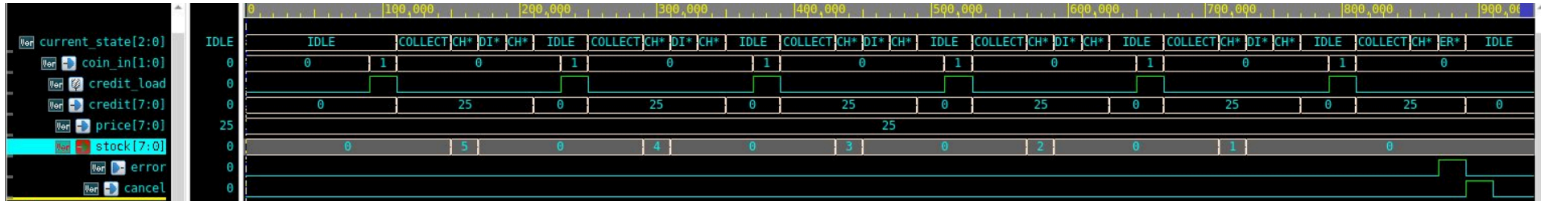
- **Cenário 2: Erro por crédito insuficiente**



- **Cenário 3: Cancelamento da operação com devolução**



- **Cenário 4:** Esgotamento de stock (Após a 5ª compra)



6. Saída do Testbench

```
=====
INICIO DA SIMULACAO - VENDING MACHINE FSM
=====
VCD+ Writer X-2025.06-SP2_Full64 Copyright (c) 1991-2025 by Synopsys Inc.
[INFO] Sistema reiniciado.

--- Iniciando Cenario 1: Compra com Troco ---
[PASS] Cenario 1: Troco correto (25 centimos).

--- Iniciando Cenario 2: Credito Insuficiente ---
[PASS] Cenario 2: Erro detetado corretamente.

--- Iniciando Cenario 3: Cancelamento da Operacao ---
[PASS] Cenario 3: Dinheiro devolvido e regressou ao IDLE.

--- Iniciando Cenario 4: Esgotamento de Stock ---
[INFO] 5 cafes comprados (Stock de cafe deve ser 0 agora).
[PASS] Cenario 4: Venda bloqueada por falta de stock.

=====
RESULTADO FINAL: >>> PASS <<<
Todos os requisitos do testbench foram cumpridos!
=====
```

7. Resultados de Síntese

Compile_ultra -np_autoungroup

Período	Frequência	Slack(ns)	Área	Timing
20 ns	50 MHz	16.30 (MET)	873.478582	0.20
18 ns	55.5 MHZ	14.30 (MET)	873.478582	0.20
16 ns	62.5 MHZ	12.30 (MET)	873.478582	0.20

14 ns	71.4 MHZ	10.30 (MET)	873.478582	0.20
12 ns	83.3 MHZ	8.30 (MET)	873.478582	0.20
10 ns	100 MHZ	6.30 (MET)	873.478582	0.20
8 ns	125 MHZ	4.30 (MET)	873.478582	0.20
6 ns	166.7 MHZ	2.30 (MET)	873.478582	0.20
4 ns	250 MHZ	0.30 (MET)	873.478582	0.20
2 ns	500 MHZ	-1.70 (VIOLATED)	873.478582	0.20

```
Compile_ultra --area_high_effort_opto
set_max_area 0
```

Período	Frequência	Slack(ns)	Área	Timing
20 ns	50 MHz	16.46 (MET)	238.049635	0.04
18 ns	55.5 MHZ	14.46 (MET)	238.049635	0.04
16 ns	62.5 MHZ	12.46 (MET)	238.049635	0.04
14 ns	71.4 MHZ	10.46 (MET)	238.049635	0.04
12 ns	83.3 MHZ	8.46 (MET)	238.049635	0.04
10 ns	100 MHZ	6.46 (MET)	238.049635	0.04
8 ns	125 MHZ	4.46 (MET)	238.049635	0.04
6 ns	166.7 MHZ	2.46 (MET)	238.049635	0.04
4 ns	250 MHZ	0.46 (MET)	238.049635	0.04
2 ns	500 MHZ	-1.54 (VIOLATED)	238.049635	0.04

```
Compile_ultra
```

Período	Frequência	Slack(ns)	Área	Timing
20 ns	50 MHz	16.35 (MET)	833.621482	0.15
18 ns	55.5 MHZ	14.35 (MET)	833.621482	0.15
16 ns	62.5 MHZ	12.35 (MET)	833.621482	0.15
14 ns	71.4 MHZ	10.35 (MET)	833.621482	0.15

12 ns	83.3 MHZ	8.35 (MET)	833.621482	0.15
10 ns	100 MHZ	6.35 (MET)	833.621482	0.15
8 ns	125 MHZ	4.35 (MET)	833.621482	0.15
6 ns	166.7 MHZ	2.35 (MET)	833.621482	0.15
4 ns	250 MHZ	0.35 (MET)	833.621482	0.15
2 ns	500 MHZ	-1.65 (VIOLATED)	833.621482	0.15

8. Análise do Projeto e Caminho Crítico

8.1. Qual é o caminho crítico do design? Quais módulos ele atravessa?

- O caminho crítico do design original-se no `u_control/current_state_reg[0]/CLK` e termina na porta da saída externa `error`. Ele atravessa o módulo `unit_control`, passando pelo flip-flop de estado e pela lógica combinacional de descodificação da saída

8.2. Como a área total variou à medida que o período foi reduzido? Explique o motivo.

- A área manteve-se constante (873.47 μm^2) em todas as iterações. Isso ocorreu porque o atraso intrínseco do circuito é muito pequeno (0.20 ns). Como as portas lógicas padrão já eram suficientemente rápidas, o Design Compiler não precisou trocá-las por células maiores para conseguir fechar o timing.

8.3. A partir de qual frequência o Design Compiler não conseguiu mais fechar o timing? O que acontece com o slack nesse ponto?

- A Partir da frequência de 500 MHZ (período de 2 ns). Nesse momento, o *Slack* tornou-se negativo (-1.70 ns, reportado como VIOLATED), indicando que o sinal não chega ao destino a tempo.

8.4. Considerando a FSM e o caminho de dados do projeto, qual estado ou transição você acredita estar no caminho crítico, e por quê?

- A permanência no estado de **ERROR** é o caminho crítico. Como `error` é uma saída de Moore, o atraso reflete o tempo exato para o sinal sair do flip-flop de estado, atravessar a lógica que identifica o Erro (`3'b101`) e chegar diretamente ao pino de saída do chip.

8.5. Que modificação no RTL você proporia para reduzir o caminho crítico sem alterar o comportamento funcional do design?

- Eu proporia **registrar as saídas da FSM** (transformando-a numa *Output Registered FSM*). Desta forma, o sinal de `error` sairia diretamente de um Flip-Flop para o pino de saída, eliminando o tempo de atraso provocado pelas

portas combinacionais intermédias e permitindo uma maior frequência máxima de operação.

9. Conclusão: Dificuldades Encontradas e Lições Aprendidas

1. Compreender a importância de manter uma separação rígida entre o *Datapath* e a Unidade de Controlo. Vi na prática que isolar a máquina de estados das operações matemáticas (como o comparador e o subtrator) facilitou muito o design modular. Isso deixou o código mais organizado e tornou a detecção de erros bem mais rápida, pois foi possível testar cada bloco isoladamente.
2. A necessidade de sempre observar os sinais pelo Verdi. Aprendi que apenas ler o código não é suficiente. Olhar as formas de onda (*waveforms*) é de extrema importância para entender o funcionamento real do circuito no tempo, ver as transições de estado exatas e conseguir fazer a detecção de possíveis falhas de sincronização que passariam despercebidas de outra forma.
3. Perceber como a frequência do relógio afeta o hardware na prática. Fazer as várias sínteses no *Design Compiler* e ir reduzindo o tempo do *clock* até o *slack* ficar negativo ajudou muito a entender o que realmente é o caminho crítico e como o circuito possui um limite físico máximo de velocidade ditado pelas portas lógicas.